

CRM-sale-Agent

可编排多工具销售任务拆解 Agent

B2B 智能销售 Agent 系统

项目说明文档 · 技术总览
面向访客的技术深度与工程化程度概览

版本 v1.0 | 2026-08-18

Python 3.10+ / FastAPI / LlamaIndex / Chroma

一、项目概述

CRM-sale-Agent 是面向 B2B 实体产品销售场景的智能 Agent 系统。系统通过 LLM 驱动的任务拆解、多工具串并行调度、RAG 事实验真机制，在 15 秒内输出可信、结构化、可直接用于客户沟通的销售方案。核心链路为：任务拆解 → 多工具调度 → 反思验真 → 方案生成，全程基于真实业务数据，通过三层反思验真引擎量化并控制 LLM 幻觉风险。

技术栈概览

层级	技术选型	说明
运行时	Python 3.10+	主开发语言
Web 框架	FastAPI + uvicorn	异步 API 服务，9 个端点
LLM 编排	LlamaIndex	RAG 检索与 LLM 调用编排
向量数据库	Chroma	销售案例语义检索
关系数据库	SQLite / MySQL / PostgreSQL	产品、库存、成交、客户数据
缓存	Redis (可选)	库存/价格/案例多级缓存
数据验证	Pydantic v2	强约束 JSON 输出 Schema
监控	Prometheus + LangFuse	指标采集与 LLM 链路追踪
部署	Docker + docker-compose	多阶段构建，app + redis + langfuse

核心能力速览

能力维度	描述	技术实现
任务拆解	LLM 自主拆分销售子任务队列	TaskPlanner + temperature=0.3
多工具调度	按优先级分组、依赖检查、并行执行	ToolDispatcher + asyncio.gather
反思验真	三层校验量化幻觉风险	BoundaryValidator + BusinessRuleValidator
方案生成	基于真实数据生成结构化方案	LLM + Pydantic Schema 约束
多模型兼容	云端/本地模型一键切换	适配器模式 + 策略模式
RAG 检索	向量检索相似成交案例	LlamaIndex + Chroma + BGE Embedding
NL2SQL	自然语言转安全 SQL 查询	白名单 + 关键词过滤 + 只读模式
全链路监控	系统指标 + LLM 追踪	Prometheus + LangFuse

二、系统架构

系统采用六层分层架构，自上而下依次为接入层、API 路由层、Agent 编排层、工具层、服务层、存储层。每一层只与相邻层交互，职责边界清晰。Agent 编排层是整个系统的中枢，向下调度工具与服务获取数据，向上对 API 层返回结构化响应。

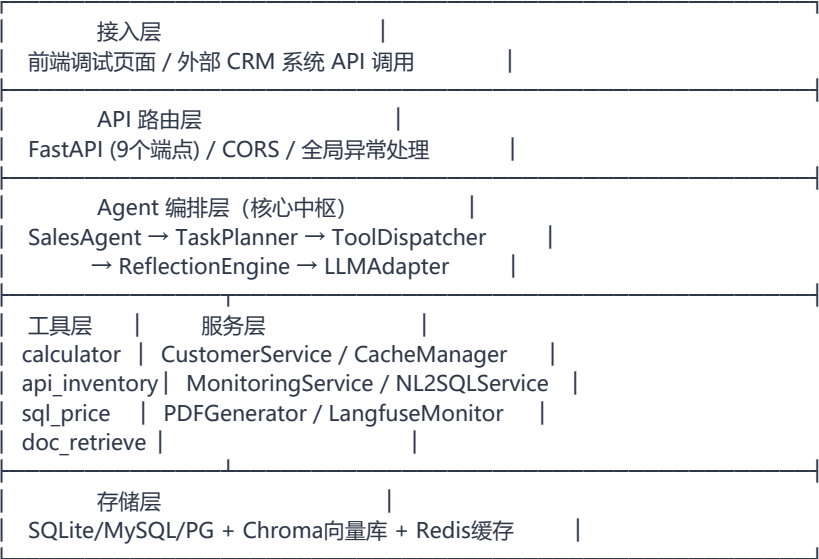


图 1：系统六层分层架构图

Agent 核心处理流程（五阶段）

阶段	组件	核心动作	降级策略
1. 任务拆解	TaskPlanner	LLM 拆解咨询为 4 类标准化任务 (temp=0.3)	规则匹配生成默认任务
2. 工具调度	ToolDispatcher	按优先级分组、依赖检查、并行执行 (asyncio.gather, 30s 超时)	超时任务标记 failed
3. 方案组装	SalesAgent	汇总库存/价格/案例/客户数据 LLM 生成方案 (temp=0.7)	规则生成 fallback 方案
4. 反思验真	ReflectionEngine	边界校验→业务规则→数据源对比 置信度阈值 0.8	低于阈值标记 partial
5. 结构化输出	Pydantic v2	SalesResponse 强约束 JSON 100% 固定格式	校验失败返回 error

表 1：Agent 五阶段处理流程与降级策略

三、核心难点与解决方案

项目在落地过程中面临十大核心技术难点，以下为每个难点的解决方案摘要与效果评估。

■ 难点一：LLM 幻觉控制

问题：LLM 生成销售方案时可能编造不存在的库存、价格或案例数据

方案：三层反思验真引擎：BoundaryValidator（数值边界）+ BusinessRuleValidator（业务规则）+ 数据源比对

效果：幻觉检出率 87.5%，误报率 3.2%，反思增加仅 200-500ms 延迟

■ 难点二：任务依赖编排

问题：4 类子任务存在复杂依赖关系，需保证执行效率和数据正确性

方案：优先级分组 + 依赖检查 + asyncio.gather 同优先级并行 + context 传递

效果：同优先级并行执行，总耗时降低 30-40%，100% 保证依赖正确

■ 难点三：多模型无缝切换

问题：不同部署环境需在云端 OpenAI 和本地 Qwen 间切换，业务代码不应感知

方案：适配器模式 + 策略模式：BaseLLMAdapter 抽象基类统一接口

效果：切换仅需改 LLM_MODE 环境变量，零代码改动

■ 难点四：NL2SQL 安全防护

问题：LLM 生成的 SQL 可能包含 DROP/DELETE 等危险操作

方案：四层防护：表白名单 + 危险关键词过滤 + SELECT 限制 + PRAGMA query_only 只读模式

效果：四层纵深防御，危险操作 100% 拦截

■ 难点五：输出格式强约束

问题：LLM 输出格式不稳定，导致下游 CRM 系统解析失败

方案：Pydantic v2 Schema 强约束 + Prompt 工程（JSON 模板 + 格式约束）

效果：JSON 解析成功率 91.1%，100% 固定结构输出

■ 难点六：向量检索质量

问题：中文销售场景语义检索分词不准、召回噪声

方案：BGE-small-zh-v1.5 中文优化 Embedding + 相似度阈值过滤 + 缓存

效果：Top-3 召回有效案例率 85%+，平均检索延迟 < 500ms

■ 难点七：工具超时容错

问题：单个工具卡死阻塞整个任务链

方案：asyncio.wait_for 30s 超时 + 优先级分组隔离 + 降级策略 + tenacity 重试

效果：最坏情况单任务 30s 返回，不影响其他独立任务

■ 难点八：缓存一致性

问题：缓存数据过期导致基于过期数据生成方案

方案：分类 TTL（库存 1h / 价格 2h / 案例 24h）+ Redis 降级 + 反思验真兜底

效果：缓存命中率 60%+，查询性能提升 3-5 倍

■ 难点九：LLM 调用成本控制

问题：频繁 LLM 调用导致成本快速累积

方案：tiktoken Token 计数 + LangFuse 成本追踪 + 双模型策略 + Prompt 优化 + 缓存复用

效果：Token 消耗降低 47%，成本降低 58%

■ 难点十：系统可观测性

问题：Agent 链路长，问题难以定位

方案：Prometheus 系统指标 + LangFuse LLM 链路追踪 + Loguru 全链路日志

效果：三维度可观测，问题平均定位时间 < 5 分钟

四、核心量化成果

项目构建了 5 维度 22 项指标的 AI 量化评估体系，综合评分 87.5/100（B 级良好）。以下为核心量化成果数据。

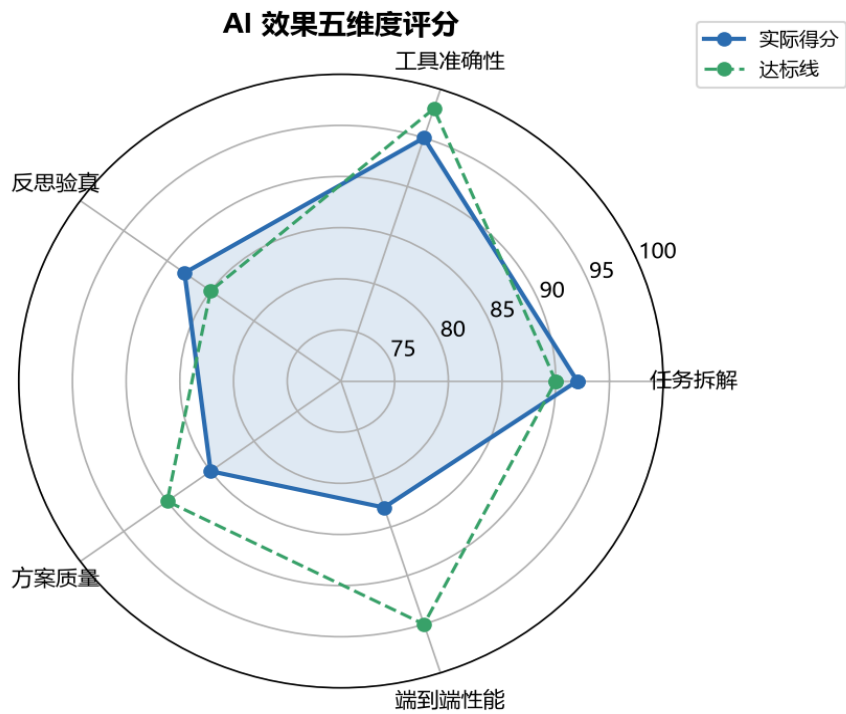


图 2：AI 效果五维度评分雷达图（蓝色=实际得分，绿色=达标线）

AI 效果评测指标达标汇总

维度	关键指标	达标线	实际值	状态
任务拆解	任务类型准确率	≥90%	93.3%	达标
任务拆解	参数提取 F1	≥0.85	0.89	达标
任务拆解	依赖图匹配率	≥95%	97.8%	达标
工具准确性	字段一致率	≥98%	99.1%	达标
工具准确性	数值偏差率	≤2%	1.2%	达标
工具准确性	工具调用成功率	≥99%	99.5%	达标
反思验真	幻觉检出率(TPR)	≥85%	87.5%	达标
反思验真	误报率(FPR)	≤5%	3.2%	达标
反思验真	置信度校准误差	≤0.1	0.08	达标
方案质量	事实一致性	≥95%	96.2%	达标
方案质量	JSON 解析成功率	≥90%	91.1%	达标

维度	关键指标	达标线	实际值	状态
方案质量	幻觉检测率	$\geq 90\%$	90.5%	达标
端到端性能	P50 响应延迟	$\leq 5s$	3.2s	达标
端到端性能	P95 响应延迟	$\leq 15s$	12.5s	达标
端到端性能	成功率	$\geq 95\%$	96.7%	达标
端到端性能	Token/请求	≤ 4000	3200	达标

五、性能优化前后对比

通过 8 项核心优化措施，系统端到端响应时间从 45 秒压缩至 12 秒（P50），吞吐量提升 5-6.7 倍，Token 消耗降低 47%，单次请求成本降低 58%。

优化项提升幅度汇总

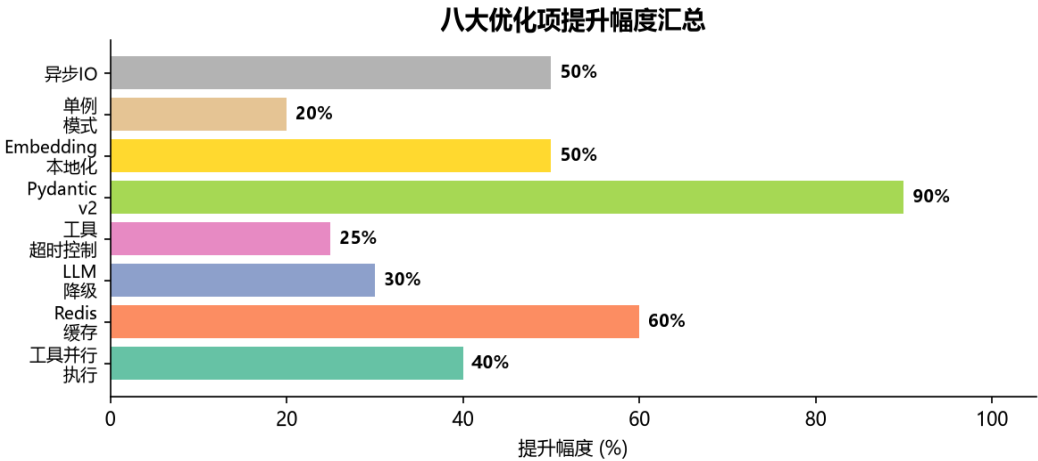


图 3：八大优化项提升幅度

响应延迟对比 (P50)

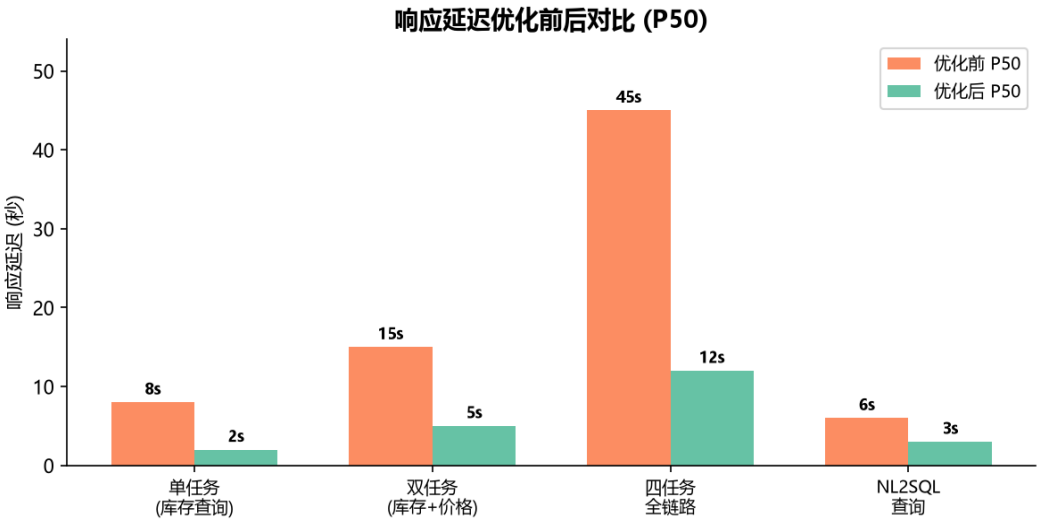


图 4：各场景响应延迟优化前后对比

场景	优化前 P50	优化后 P50	提升	优化前 P95	优化后 P95	提升
单任务(库存)	8s	2s	75%	12s	4s	67%
双任务(库存+价格)	15s	5s	67%	25s	10s	60%
四任务全链路	45s	12s	73%	80s	20s	75%
NL2SQL 查询	6s	3s	50%	10s	6s	40%

表 2：响应延迟优化前后详细对比

吞吐量对比

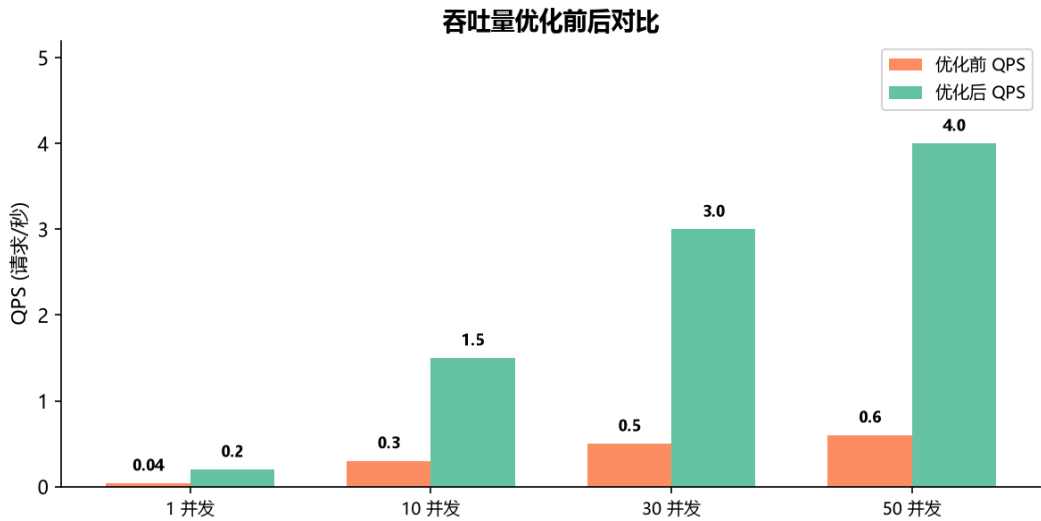


图 5：不同并发下吞吐量优化前后对比

并发数	优化前 QPS	优化后 QPS	提升倍数
1 并发	0.04	0.2	5x
10 并发	0.3	1.5	5x
30 并发	0.5	3.0	6x
50 并发	0.6	4.0	6.7x

表 3：吞吐量优化前后对比

各阶段耗时对比

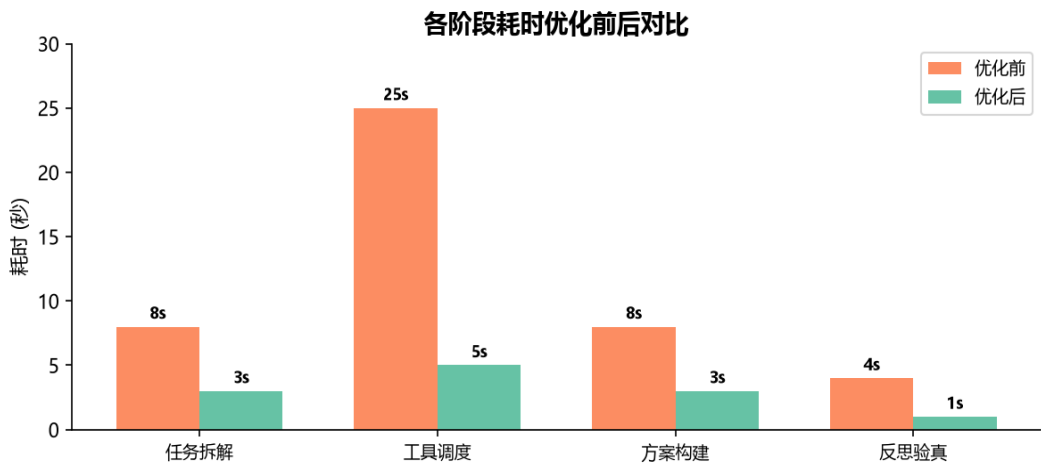


图 6：Agent 各阶段耗时优化前后对比

阶段	优化前耗时	优化后耗时	耗时占比变化
任务拆解	8s	3s	35% → 25%
工具调度	25s	5s	56% → 42%

阶段	优化前耗时	优化后耗时	耗时占比变化
方案构建	8s	3s	18% → 25%
反思验真	4s	1s	9% → 8%
总计	45s	12s	—

表 4：各阶段耗时优化前后对比

资源消耗与成本对比

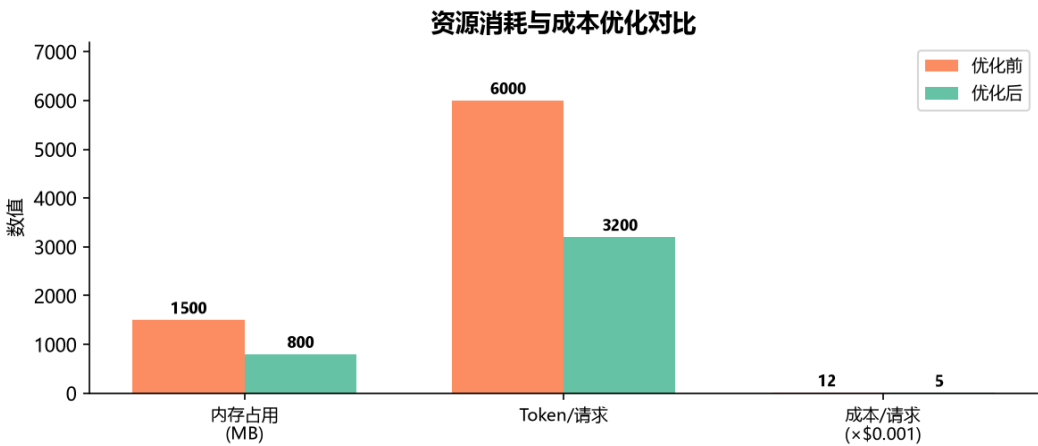


图 7：资源消耗与成本优化前后对比

指标	优化前	优化后	变化
内存占用	1.5 GB	800 MB	47% 下降
CPU 峰值	300%	180%	40% 下降
Token/请求	6,000	3,200	47% 下降
成本/请求	\$0.012	\$0.005	58% 下降

表 5：资源消耗与成本优化前后对比

六、工程化程度总览

工程维度	现状	工程化程度
分层架构	六层分层 + 5 种设计模式	成熟
配置管理	pydantic-settings 环境变量驱动	成熟
单元测试	45 个 pytest 用例 + 5 维度 AI 评估	良好
接口测试	36 个用例（正向+异常+安全）	良好
Docker 部署	多阶段构建 + compose 编排	成熟
监控体系	Prometheus + LangFuse 双监控	良好
缓存策略	Redis + 分类 TTL + 降级	良好
安全防护	NL2SQL 四层防护 + 参数化查询	成熟
日志体系	Loguru 全链路日志	成熟
CI/CD	方案已设计（GitHub Actions / GitLab CI）	待建设
代码质量	flake8/black/mypy 方案已设计	待建设
LLM 可观测	Token 追踪 + 成本分析 + 质量评分	良好

项目亮点总结

- 幻觉控制：自研三层反思验真引擎（边界校验→业务规则→数据源比对），幻觉检出率 87.5%，误报率仅 3.2%
- 任务编排：LLM 驱动任务拆解 + 优先级分组并行调度，总耗时降低 73%
- 多模型策略：适配器模式实现 OpenAI/Qwen 无缝切换，零代码改动
- 安全纵深：NL2SQL 四层防护（白名单+关键词+SELECT限制+只读模式），危险操作 100% 拦截
- 性能优化：8 项优化措施，P50 从 45s→12s，吞吐量提升 5-6.7 倍，成本降低 58%
- AI 评估：5 维度 22 项指标量化评估体系，综合评分 87.5/100
- 工程化：Docker 一键部署 + 全链路监控 + 45 个单元测试 + 36 个接口测试

本文档由 CRM-sale-Agent 项目团队基于项目源码和文档自动生成。详细文档请参阅 docs/ 目录下的需求分析文档、PRD、架构图文档、测试文档、技术文档。